

FitBuddy – AI Fitness Plan Generator using Gemini Models

5. Backend, Routes & API Processing

Nutrition Tip Generation: `generate_nutrition_tip_with_flash()` uses Gemini Flash to generate a practical tip related to the user's fitness goal.

Workout Plan Update via Feedback: `update_workout_plan()` passes original plan + user feedback to Gemini Pro and returns a revised plan.

All responses are returned to the user through `result.html`, maintaining clarity using `<pre>` blocks for formatting.

Workout generation route Nutrition/Recovery tip generation route

Nutrition/Recovery tip generation route

Saving user info and plan generation route

Submit Feedback and update plan route

Milestone 4. Frontend Development

Milestone 4 focuses on developing a user-friendly and visually structured interface for FitBuddy, the AI-powered workout generator. This involves building responsive and interactive web pages using HTML, CSS, and Jinja2 templating. Each template is dynamically rendered through FastAPI, ensuring users can easily submit input, receive personalized plans, provide feedback, and allow administrators to manage data effectively. This milestone guarantees an engaging and intuitive fitness planning experience for all users.

Activity 4.1 Designing and Developing the User Interface

Set Up the Base HTML Structure

Developed `index.html` as the main entry point for user input.

Structured the form to capture:

username, user_id, age, weight, goal, and intensity.

Each field is properly labeled and grouped using semantic HTML.

The design uses a gym-themed background image, Google Fonts (Roboto), and bold typography to create a clean, fitness-oriented interface.

Navigation is kept minimal for focus, and user flow directs clearly from input to output.

Design a Responsive Layout Using CSS

Embedded CSS styles were applied directly in each HTML file.

Layouts use Flexbox to center content both vertically and horizontally.

Media queries ensure mobile responsiveness.

Buttons have consistent styling with hover effects.

Input fields and result blocks have adequate padding, shadows, and rounded corners for modern aesthetics.

Overall color scheme supports readability on a dark background.

Create Separate Pages for Each Core Functionality

Three Jinja2-powered HTML templates were created under /templates:

index.html: A user input form with fields for fitness goal, intensity, and basic personal information.

result.html: Displays the 7-day AI generated workout plan, a nutrition tip, and accepts user feedback.

all_users.html: Admin panel to view and delete users and review both original and updated plans.

Templates are modular, and UI transitions smoothly between steps.

Forms are submitted to specific FastAPI endpoints via POST methods

Activity 4.2 Creating Dynamic Templates with FastAPI's Jinja2

Integrate Jinja2 Templating for Dynamic Content Rendering

Used FastAPI's Jinja2Templates for template rendering:

```
from fastapi.templating import Jinja2Templates
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
TEMPLATE_DIR = os.path.join(BASE_DIR, "templates")
```

```
templates = Jinja2Templates(directory=TEMPLATE_DIR)
```

Each route uses TemplateResponse to render the proper HTML with passed context:

This allows dynamic plan and nutrition tip display after form submission.

Bind Backend Data to HTML Templates

- index.html

Form elements use name="..." to bind to backend Form(...) parameters.

Submits data to /generate-workout.

- result.html

Uses:

{{ username }}, {{ goal }}, {{ intensity }}, etc. to show user data.

{{ workout_plan }} and {{ nutrition_tip }} to show AI-generated results.

Includes a feedback form submitted via POST to /submit-feedback.

- all_users.html

Loops over all user records using:

Milestone 5. Deployment

Now, the focus is on deploying the FitBuddy application on a local system using FastAPI and Uvicorn. The app uses HTML templates rendered with Jinja2, alongside AI services powered by Gemini. This milestone ensures all environment dependencies are correctly installed, and the app runs smoothly on a local server, preparing it for future deployment on cloud platforms.

Activity 5.1: Preparing the Application for Local Deployment

Set up a virtual environment

Create a virtual environment to manage dependencies and isolate the FitBuddy project from other environments.

Activate the environment and install all necessary dependencies listed in the requirements.txt file, including FastAPI, Uvicorn, SQLAlchemy, and Google Generative AI SDK.

```
python -m venv venv
```

```
source venv/bin/activate # On Windows: venv\Scripts\activate
```

```
pip install -r requirements.txt
```

Configure environment variables

Use environment variables to store sensitive data such as the Gemini API Key.

These variables can be set directly in the terminal or added to a .env file which is loaded into the application using python-dotenv.

GOOGLE_API_KEY=your_gemini_api_key_here

Frontend-Backend Rendering via Jinja2

All user interactions are facilitated via Jinja2-rendered templates:

index.html: Collects user inputs

result.html: Displays the generated AI plan

all_users.html: Lists all user records for admin view

Activity 5.2: Testing and Verifying Local Deployment

Access the Application Locally

The FastAPI development server must be started using the command (uvicorn main:app --reload). Once the server is running, the application can be accessed through any web browser by navigating to <http://127.0.0.1:8000>. This URL loads the main user interface where individuals can input their details and generate personalized fitness plans. Additionally, the interactive API documentation provided by FastAPI can be viewed at <http://127.0.0.1:8000/docs>, allowing developers to explore and test available endpoints.

Exploring Website's web pages:

Home page:

Description: The homepage of FitBuddy – AI Workout Generator features a clean and engaging user interface designed to provide a seamless experience for users seeking personalized fitness plans. The form prominently displayed in the center allows users to enter essential information including:

Name

User ID

Age

Weight (kg)

Fitness Goal (e.g., weight loss, flexibility)

Workout Intensity (Low, Medium, High)

Once users submit this data, they can click the "Generate Plan" button, which triggers the AI-powered backend to generate a customized workout and nutrition plan tailored to their goals and fitness profile. The visual appeal of the background image featuring an active individual reinforces the fitness-centric theme of the application, motivating users to take charge of their health journey.

This intuitive design ensures users of all experience levels can quickly get started with minimal effort.

Personalized workout generation page: